Full Stack Online Learning Platform Using Spring Boot Docker and Kubernetes

Project Phase-I Report submitted to Osmania University, Hyderabad, in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

In

Information Technology

By

Mr. Syed Riyan Mohiuddin	**(160923737156)**
Mr. Mohammed Ajmal Shaik	**(160923737132)**
Mr. Shaikh Faizan Ahmed	**(160923737130)**

Under the Guidance of

Ms. Khutaija Abid

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

(Affiliated to Osmania University) Himayathsagar, Hyderabad,500091

A.Y. 2026-2027



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091
Department of Information Technology

CERTIFICATE

This is to certify that the Project Phase-I report entitled **Full Stack Online Learning Platform Using Spring Boot, Docker and Kubernetes** being submitted by **Syed Riyan Mohiuddin (160923737156), Mohammed Ajmal Shaik (160923737132) and Shaikh Faizan Ahmed (160923737130)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology** is a record of Bonafide work carried out by them.

Internal Project Guide
Ms. Khutaija Abid
Assistant Professor

Dr. K. Nagi Reddy
H.O.D, I.T

External Examiner



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)
(Affiliated to Osmania University)
Himayathsagar, Hyderabad, 500091

Department of Information Technology

DECLARATION BY THE CANDIDATES

We, **Syed Riyan Mohiuddin (160923737156), Mohammed Ajmal Shaik (160923737132) and Shaikh Faizan Ahmed (160923737130)**, hereby declare that the Project Phase-I report entitled **Full Stack Online Learning Platform Using Spring Boot, Docker and Kubernetes** is carried out under the guidance of **Ms. Khutaija Abid** Assistant Professor, Department of Information Technology, **Lords Institute of Engineering and Technology**, **Hyderabad** is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Information Technology.**

This is a record of Bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

Syed Riyan Mohiuddin (160923737156)
Mohammed Ajmal Shaik (160923737132)
Shaikh Faizan Ahmed (160923737130)

ACKNOWLEDGMENT

We are very pleased to present this thesis of our project. This period of our student life has been truly rewarding a number of people were of immense help to us during the course of our research and the preparation of our thesis.

First, we wish to thank **GOD Almighty** who created heavens and earth, who helped us in completing this project and we also thank our parents who encouraged us in this period of research.

We would like to express our sincere gratitude to **The Management** of **Lords Institute of Engineering and Technology, Hyderabad** for providing us with an excellent platform and resources to successfully carry out this project.

We would also like to thank **Dr. Ravi Kishore Singh, Principal, L.I.E.T.** for extending his help in overall completion of this project work.

We would like to express my deep sense of gratitude **to Dr. K. Nagi Reddy, Professor & HOD**, **Information Technology, Lords Institute of Engineering & Technology, Hyderabad,** for his encouragement and cooperation throughout the project.

We would like to thank **Ms. Khutaija Abid, Assistant Professor, Dept. of Information Technology, Lords Institute of Engineering & Technology**, Hyderabad, our project internal guide, for her guidance and help. Her insight during the course of our research and regular guidance were invaluable to us.

Lastly, we acknowledge the support of our faculty members, friends, and family, who inspired and encouraged us throughout this journey.

Syed Riyan Mohiuddin (160923737156)
Mohammed Ajmal Shaik (160923737132)
Shaikh Faizan Ahmed (160923737130)

Vision of the Institute

Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute

- To impart quality professional education that meets the needs of present and emerging technological world.
- To strive for student achievement and success, preparing them for life, career and leadership.
- To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
- To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
- To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

PRINCIPAL
Principal
Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.T.S.

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY

Approved by AICTE/Affiliated to Osmania University/Estd.2003.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision of the Department:

To be a frontier in Information Technology focusing on quality professional education, innovation and employability.

Mission of the Department:

DM1: Adopt innovative teaching learning processes to achieve stated outcomes and sustained performance levels.

DM2: Provide learner centric environment to enrich knowledge, skills and innovation through industry academia collaborations.

DM3: Train the concerned stakeholders on latest technologies to meet the industry needs.

DM4: Inculcate leadership, professional, interpersonal skills and ethical values to address the contemporary societal issues.

Note: DM: Department Mission

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Educational Objectives (PEOs):

PEO1	Be a competent software engineer in IT and allied industry providing viable solutions.
PEO2	Exhibit professionalism, engage in lifelong learning and adopt to changing industry and societal needs.
PEO3	Be effective as an individual and also cohesive in a group to execute multidisciplinary projects.
PEO4	Pursue higher studies in Information Technology with emphasis on competency and innovation.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technolo
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Outcomes (POs):

Engineering Graduates will be able to:

S. No.	Program Outcomes (POs)
1.	**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2.	**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3.	**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5.	**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
7.	**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8.	**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9.	**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
10.	**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11.	**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

B.E Information Technology Program Specific Outcomes (PSOs):

PSO1	**Professional Skills:** Identify, analyze requirements, design and implement computer programs in the area of artificial intelligence, machine learning, big data analytics and networking of varying complexity.
PSO2	**Problem-Solving Skills:** Deliver quality service to IT industry and provide solutions to real-life problems applying modern computer languages, software methods and tools.

Head of the Department
Head of the Department
Information Technology
Lords Institute of Engineering & Technology
Himayath Sagar, Hyderabad-91

Course Outcomes: Project Phase- I

After completion of the project, the student will be able to

S.No	Description	Blooms Taxonomy Level
PP.I.CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to the real-world problems.	BTL3
PP.I.CO2	Evaluate different solutions based on economic and technical feasibility	BTL4
PP.I.CO3	Effectively plan a project and confidently perform all aspects of project management.	BTL5
PP.I.CO4	Demonstrate effective written and oral communication skills	BTL4
PP.I.CO5	Prepare the documentation report and perform the presentation of the project work	BTL3

LORDS INSTITUTE OF ENGINEERING &TECHNOLOGY
Approved by AICTE/Affiliated to Osmania University/Estd.2002.
Accredited 'A' grade by NAAC
(UGC Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Articulation Matrix:

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific Outcomes (PSOs):

Course Outcomes (COs)	Program Outcomes (POs)											Program Specific Outcomes (PSOs)	
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PSO1	PSO2
PP.I.CO1	3	2	2	-	2	2	-	-	3	-	-	2	2
PP.I.CO2	3	3	2	3	2	-	2	-	3	-	-	-	2
PP.I.CO3	3	3	3	-	3	3	1	2	2	3	2	3	-
PP.I.CO4	3	2	2	3	3	3	3	2	-	-	2	3	3
PP.I.CO5	3	3	3	3	3	3	3	2	3	3	-	3	3
AVG	**3**	**2.6**	**2.5**	**3**	**2.6**	**2.7**	**2.2**	**2**	**2.9**	**3**	**2**	**2.7**	**2.5**

Level:

1- Low correlation (Low), 2- Medium correlation (Medium), 3-High correlation (High)

SDG Mapping:

SDG	Mapped Indicator	SDG	Mapped Indicator	SDG	Mapped Indicator
1 NO POVERTY		7 AFFORDABLE AND CLEAN ENERGY		13 CLIMATE ACTION	
2 ZERO HUNGER		8 DECENT WORK AND ECONOMIC GROWTH		14 LIFE BELOW WATER	
3 GOOD HEALTH AND WELL-BEING		9 INDUSTRY, INNOVATION AND INFRASTRUCTURE	✓	15 LIFE ON LAND	
4 QUALITY EDUCATION	✓	10 REDUCED INEQUALITIES		16 PEACE, JUSTICE AND STRONG INSTITUTIONS	
5 GENDER EQUALITY		11 SUSTAINABLE CITIES AND COMMUNITIES		17 PARTNERSHIPS FOR THE GOALS	
6 CLEAN WATER AND SANITATION		12 RESPONSIBLE CONSUMPTION AND PRODUCTION			

ABSTRACT

The rapid growth of digital education has created a need for flexible, accessible, and scalable online learning platforms that allow students and instructors to interact through a unified digital environment. This project proposes the design and development of a **Full-Stack Online Learning Platform Using Spring Boot, Docker and Kubernetes.** The platform is designed to provide students with an efficient environment for discovering courses, enrolling in courses, accessing learning materials, completing quizzes, and tracking their learning progress.

The backend of the system is developed using Spring Boot, which provides RESTful APIs for user authentication, course management, enrollment, lesson management, quizzes, and progress tracking. The platform supports different user roles, including students, instructors, and administrators, with role-based access to their respective functionalities. A web-based frontend provides an interactive interface through which users can access the platform's services, while a relational database is used to securely store user, course, enrollment, assessment, and progress-related information.

To improve portability and deployment efficiency, the application components are packaged into Docker containers. Kubernetes is used for container orchestration, enabling efficient deployment, service management, scaling, and improved availability of the application. The system follows a modular architecture that allows its components to be independently managed and deployed.

The proposed platform demonstrates the integration of modern full-stack development with containerization and orchestration technologies to create a scalable and maintainable online learning environment. The project provides practical implementation of Spring Boot, RESTful APIs, database management, authentication, Docker, Kubernetes, and cloud-oriented deployment, making the system suitable for modern educational applications.

Keywords: Online Learning, Full-Stack Development, Spring Boot, REST API, Docker, Kubernetes, eLearning, Containerization, Cloud Deployment

CONTENTS

Certificate i

Declaration ii

Acknowledgement iii

Vision and Mission of the Institute iv

Vision and Mission of the Department v

Program Educational Objectives vi

Program Outcomes vii

Program Specific Outcomes viii

Course Outcomes ix

Course Articulation Matrix x

Abstract xi

Contents xii

List of Figures xv

List of Tables xvi

CHAPTER-1: INTRODUCTION 1

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Project 2

1.5 Existing System 3

1.6 Proposed System 3

1.7 Advantages 3

CHAPTER-2: LITERATURE SURVEY 4

2.1 Introduction 4

2.2 Online Learning Management Systems 4

2.3 Full-Stack Web Development 4

2.4 Spring Boot 4

2.5 Frontend Technologies 5

2.6 Relational Database Management System 6

2.7 Docker 6

2.8 Kubernetes 6

2.9 Security Technologies 7
2.10 Comparative Analysis 7
2.11 Summary 8
CHAPTER-3: SYSTEM ANALYSIS AND REQUIREMENTS 9
3.1 Introduction 9
3.2 Functional Requirements 9
3.3 Non-Functional Requirements 10
3.4 Hardware Requirements 11
3.5 Software Requirements 12
3.6 User Roles 12
3.7 Feasibility Study 13
3.8 Use Case Overview 13
3.9 Summary 13
CHAPTER-4: SYSTEM DESIGN AND ARCHITECTURE 14
4.1 Introduction 14
4.2 System Architecture 14
4.3 Major System Modules 15
4.4 Database Design 17
4.5 REST API Design 18
4.6 Deployment Architecture 18
4.7 Summary 19
CHAPTER-5: IMPLEMENTATION AND TESTING 20
5.1 Introduction 20
5.2 Development Environment Setup 20
5.3 Backend Implementation 20
5.4 Frontend Implementation 22
5.5 Database Implementation 23
5.6 Docker Implementation 24
5.7 Kubernetes Implementation 25
5.8 Testing Strategy 26
5.9 Sample Test Cases 27
5.10 Performance and Deployment Evaluation 27
5.11 Summary 28

CHAPTER-6: RESULTS AND FUTURE ENHANCEMENTS **29**

6.1 Introduction 29

6.2 Expected Results 29

6.3 Result Evaluation 30

6.4 Project Contributions 30

6.5 Limitations 31

6.6 Future Enhancements 31

CONCLUSION **32**

REFERENCES **33**

List of Figures

SI.no	Fig.no	Name of the Figure	Page No
1	4.1	System Architecture	15
2	4.2	Deployment Architecture	19
3	5.1	Backend Architecture	21
4	5.2	Frontend Architecture	22
5	5.3	Database Architecture	23
6	5.4	Docker Architecture	24
7	5.5	Kubernetes Architecture	25

List of Tables

SI.no	Table.no	Name of the Table	Page No
1	2.1	Comparative Analysis	7
2	3.1	Hardware Requirements	11
3	3.2	Software Requirements	12
4	4.1	Database Design	17
5	4.2	REST API Design	18
6	5.1	Sample Test Cases	27
7	6.1	Result Evaluation	30

CHAPTER-1
INTRODUCTION

1.1 Introduction

The Full Stack Online Learning Platform Using Spring Boot, Docker and Kubernetes is a web-based application designed to provide an integrated digital learning environment for students, instructors, and administrators. The platform enables learners to access online courses, watch learning materials, enroll in courses, complete assessments, track their progress, and interact with instructors.

The system uses Spring Boot to develop scalable backend REST APIs, a modern web frontend to provide an interactive user interface, and a relational database to manage users, courses, enrollments, assessments, and learning progress.

Docker is used to package the application and its dependencies into containers, ensuring consistent execution across development, testing, and production environments. Kubernetes manages containerized application deployment, scaling, service discovery, and recovery from container failures.

The proposed platform combines full-stack development practices with containerization and orchestration technologies to create a reliable, maintainable, and scalable online learning solution.

1.2 Problem Statement

Traditional learning management systems may face challenges related to accessibility, scalability, deployment complexity, learner progress tracking, and centralized course management.

In applications deployed without containerization, differences between development and production environments can lead to configuration problems. Manual deployment and scaling can also become difficult when the number of users increases.

The proposed system addresses these challenges by developing a centralized online learning platform with structured course management, automated learning progress tracking, containerized deployment, and Kubernetes-based orchestration.

1.3 Objectives

The primary objectives are:

1. To develop a responsive full-stack online learning application.
2. To provide secure registration, login, and role-based access.
3. To enable instructors to create, update, and manage courses.
4. To allow students to enroll in courses and access learning materials.
5. To implement quizzes, assessments, and progress tracking.
6. To develop REST APIs using Spring Boot.
7. To store application data using a relational database.
8. To containerize the frontend and backend using Docker.
9. To deploy and manage application containers using Kubernetes.
10. To evaluate application functionality, performance, security, and deployment reliability.

1.4 Scope of the Project

The project covers the design, development, testing, containerization, and deployment of an online learning platform.

The major functionalities include:

- User registration and authentication.
- Student, instructor, and administrator roles.
- Course creation and course enrollment.
- Learning content management.
- Quiz and assessment management.
- Student progress tracking.
- Course completion reports.
- Containerized application deployment.
- Kubernetes service management and scaling.

Advanced features such as live video conferencing, AI-based personalized recommendations, online payment integration, and automated certificate verification can be incorporated as future enhancements.

1.5 Existing System

In conventional learning systems, course information, assessments, and learner progress may be managed through separate tools or manually maintained records. Some applications also rely on monolithic deployments and manual infrastructure management.

The limitations of such systems may include:

- Limited scalability during peak usage.
- Difficulties in deploying application updates.
- Inconsistent development and production environments.
- Manual monitoring and recovery procedures.
- Limited integration between learning activities and progress reports.

1.6 Proposed System

The proposed system provides a centralized learning platform supported by a modern full-stack architecture.

The frontend communicates with Spring Boot REST APIs. The backend applies business rules, validates requests, and accesses the database through Spring Data JPA. Docker packages the application components, while Kubernetes manages their deployment and availability.

The system provides a structured workflow for managing courses, delivering learning content, recording assessments, and monitoring student progress.

1.7 Advantages

- Centralized course and learner management.
- Improved accessibility through a web interface.
- Secure access based on user roles.
- Easier deployment using Docker containers.
- Better application availability through Kubernetes.
- Horizontal scaling when supported by suitable resource configuration.

CHAPTER-2
LITERATURE SURVEY

2.1 Introduction

This chapter reviews the technologies and architectural concepts used to develop the online learning platform. It explains the role of full-stack development, RESTful services, relational databases, containerization, and container orchestration.

2.2 Online Learning Management Systems

A Learning Management System (LMS) is a software application that supports the creation, distribution, and management of educational content.

Typical LMS functionality includes course enrollment, learning material delivery, assessment management, user administration, and performance reporting.

The proposed application follows these core LMS concepts while emphasizing containerized application deployment and infrastructure scalability.

2.3 Full-Stack Web Development

Full-stack development involves implementing both the client-facing interface and server-side application logic.

The frontend is responsible for displaying course information, collecting user input, and presenting progress reports. The backend processes requests, enforces authorization, applies business rules, and interacts with persistent storage.

A REST-based interface separates the frontend from backend implementation details and allows the components to evolve independently.

2.4 Spring Boot

Spring Boot is a Java framework used to develop standalone, production-oriented applications with reduced configuration [1].

The proposed project uses Spring Boot for:

- Developing RESTful APIs.
- Implementing business services.
- Managing dependency injection.
- Integrating with relational databases.
- Supporting authentication and authorization.
- Validating incoming requests.
- Handling application exceptions.

Spring Data JPA simplifies database operations through repository interfaces [2], while Spring Security provides authentication and authorization capabilities [3].

2.5 Frontend Technologies

The frontend can be implemented using React with TypeScript or JavaScript. React supports reusable interface components and state-driven rendering [4], [5].

The main frontend screens include:

- Login and registration.
- Student dashboard.
- Instructor dashboard.
- Course catalog.
- Course details and learning content.
- Quiz interface.
- Progress and results dashboard.
- Administrator management screens.

HTML, CSS, JavaScript, and a responsive UI framework can be used to develop the user interface.

2.6 Relational Database Management System

PostgreSQL or MySQL can be used to store structured application data [6].

The database maintains information about users, courses, lessons, enrollments, assessments, questions, submissions, and student progress.

Spring Data JPA and Hibernate provide object-relational mapping between Java entities and database tables [2].

2.7 Docker

Docker is a containerization platform that packages an application and its required runtime components into a container image [7], [8].

In this project, Docker is used to:

- Build the backend application image.
- Package the frontend application.
- Define container configuration.
- Configure environment variables.
- Support repeatable deployment.
- Simplify local integration testing.

Docker Compose may be used during development to start the frontend, backend, and database together.

2.8 Kubernetes

Kubernetes is a container orchestration platform used to deploy, scale, and manage containerized applications [9], [10].

The proposed application may use the following Kubernetes resources:

- Pods: Run application containers.
- Deployments: Manage application replicas and rolling updates.
- Services: Provide stable internal communication and access.

- ConfigMaps: Store non-sensitive configuration.
- Secrets: Store credentials and other sensitive configuration.
- Ingress: Provide HTTP or HTTPS routing when an ingress controller is configured.
- Horizontal Pod Autoscaler: Adjust replicas based on configured metrics and scaling policies.

Kubernetes improves deployment management, but actual high availability and autoscaling depend on correct configuration and available cluster resources [9].

2.9 Security Technologies

Spring Security can be used with session-based authentication or JWT-based authentication. For a frontend that communicates with REST APIs, JWT-based authentication is one possible implementation [3].

Security measures include password hashing, role-based authorization, request validation, secure configuration, and protection against unauthorized access.

Database passwords, signing keys, and other sensitive values should not be embedded in source code or container images.

2.10 Comparative Analysis

Table 2.1 Comparative Analysis

Technology	Purpose	Benefit
React	Frontend development	Reusable and interactive UI
Spring Boot	Backend development	Structured REST API development
Spring Security	Application security	Authentication and authorization
Spring Data JPA	Database access	Simplified persistence operations
PostgreSQL/MySQL	Data storage	Persistent relational data
Docker	Containerization	Consistent application packaging
Kubernetes	Orchestration	Deployment and scaling management
Maven	Build management	Dependency and build automation
Git	Version control	Source tracking and collaboration

2.11 Summary

The reviewed technologies provide the foundation for building a full-stack learning application. Spring Boot and the frontend framework implement application functionality, the database maintains persistent records, and Docker and Kubernetes support deployment and operational management.

CHAPTER-3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Introduction

System analysis identifies the functional capabilities, technical requirements, user roles, and operational constraints of the proposed platform.

The requirements are divided into functional requirements and non-functional requirements to guide implementation and testing.

3.2 Functional Requirements

FR1: User Registration

The system shall allow new users to register by providing required information such as name, email address, and password.

FR2: User Authentication

The system shall validate user credentials and provide access according to the user's assigned role.

FR3: User Management

The administrator shall be able to view and manage registered users according to authorized permissions.

FR4: Course Management

Instructors shall be able to create, update, publish, and manage courses.

FR5: Course Enrollment

Students shall be able to browse available courses and enroll in published courses.

FR6: Learning Content Management

Instructors shall be able to organize courses into modules and lessons and upload or link learning resources.

FR7: Assessment Management

Instructors shall be able to create quizzes and assessments containing questions and expected answers.

FR8: Quiz Participation

Students shall be able to attempt available quizzes and submit their responses.

FR9: Progress Tracking

The system shall record lesson completion, quiz results, and course progress.

FR10: Reporting

Students shall be able to view their own learning progress, while instructors shall be able to view relevant course-level results.

FR11: Containerized Deployment

The frontend and backend shall be packaged as Docker images.

FR12: Kubernetes Deployment

The application shall support deployment using Kubernetes resources for application management, service access, and scaling.

3.3 Non-Functional Requirements

Performance: The application should respond within acceptable time limits under the expected workload.

Scalability: Backend replicas should be capable of scaling horizontally when application design, database capacity, and cluster resources permit.

Security: The system should enforce authentication, authorization, password hashing, and secure handling of secrets.

Availability: Kubernetes health probes and restart policies should support recovery from certain container failures.

Maintainability: The application should use layered architecture, modular services, and clearly defined API contracts.

Usability: The interface should be intuitive and responsive across desktop, tablet, and mobile devices.

Reliability: The system should validate input, handle exceptions, and preserve consistent database records.

Portability: Docker images should support deployment across compatible environments.

3.4 Hardware Requirements

The following are suggested development requirements, not mandatory production specifications.

Table 3.1 Hardware Requirements

Component	Suggested Requirement
Processor	Modern multi-core processor
RAM	8 GB minimum; 16 GB or more recommended for local Kubernetes
Storage	20 GB or more available space
Network	Internet connectivity for dependency downloads and image retrieval
Client device	Desktop, laptop, tablet, or smartphone

Production infrastructure should be sized according to expected users, concurrent sessions, media delivery needs, and database workload.

3.5 Software Requirements

Table 3.2 Software Requirements

Software	Purpose
Operating System	Windows, Linux, or macOS
Java 17 or 21	Backend runtime and development
Spring Boot	REST API development
Maven	Build and dependency management
React	Frontend development
Node.js and npm	Frontend build tooling
PostgreSQL or MySQL	Persistent storage
Docker Engine	Container image creation and execution
Kubernetes	Container orchestration
kubectl	Kubernetes cluster management
Git	Version control
Postman	API testing
IDE	IntelliJ IDEA, Eclipse, or VS Code

The exact Spring Boot version and Java version should be selected from a compatible supported combination.

3.6 User Roles

Administrator

Manages users, role assignments, course oversight, and system configuration.

Instructor

Creates courses, organizes lessons, prepares assessments, and reviews student performance.

Student Enrolls in courses, accesses learning materials, attempts quizzes, and monitors personal progress.

3.7 Feasibility Study

Technical Feasibility: The project can be implemented using established Java, frontend, database, containerization, and orchestration technologies.

Economic Feasibility: Development can begin with open-source frameworks and local container environments, reducing initial software licensing costs. Cloud hosting and managed Kubernetes services may introduce infrastructure costs.

Operational Feasibility: The browser-based interface can be used by students, instructors, and administrators without requiring a separate desktop installation.

Schedule Feasibility: The application can be developed incrementally, beginning with authentication and course management before introducing assessments, deployment automation, and Kubernetes scaling.

3.8 Use Case Overview

The primary use cases are:

- Register and log in.
- Manage users.
- Create and publish courses.
- Browse and enroll in courses.
- Access course lessons.
- Attempt quizzes.
- Track learning progress.
- Review assessment results.
- Deploy and monitor application workloads.

3.9 Summary

The requirements define the expected behavior of the learning platform and establish the technical foundation for architecture design and implementation.

CHAPTER-4
SYSTEM DESIGN AND ARCHITECTURE

4.1 Introduction

This chapter presents the architecture, application modules, database design, API organization, and deployment approach for the online learning platform.

The architecture separates the frontend, backend business logic, database, and container orchestration layers. This separation supports independent development, testing, deployment, and maintenance.

4.2 System Architecture

Presentation Layer

React + TypeScript/JavaScript

Student, Instructor and Admin Dashboards

Application Layer

Spring Boot REST APIs

Controllers · Services · Repositories · Security

Data Layer

PostgreSQL / MySQL

Users · Courses · Lessons · Enrollments · Results

Deployment and Infrastructure Layer

Docker Images + Kubernetes

Deployments · Services · Probes · Scaling · Configuration

The frontend sends HTTP requests to the backend REST APIs. The backend authenticates users, checks permissions, processes business operations, and retrieves or stores data in the database.

Docker packages application components into images [7]. Kubernetes runs and manages those containers [9]. Persistent database storage must be configured separately so that database records are not lost when containers are replaced.

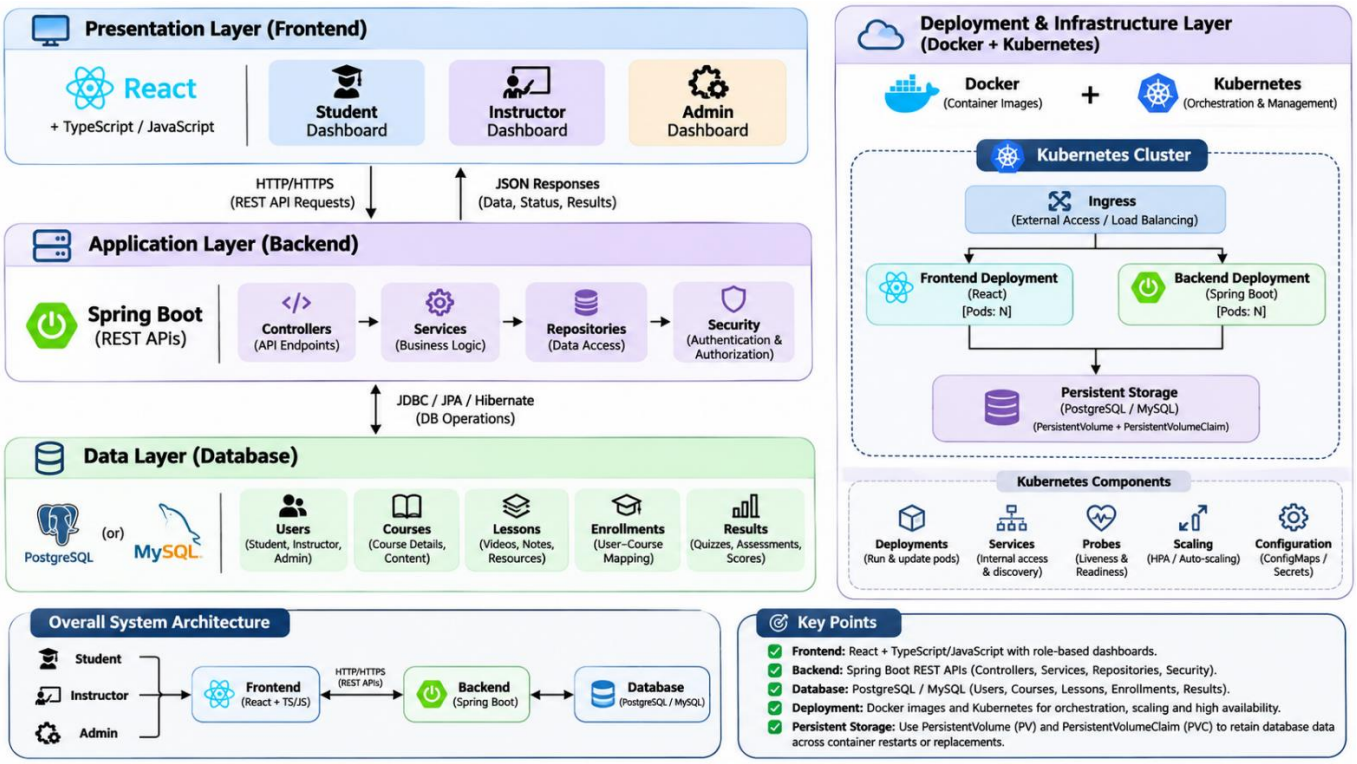


Figure 4.1 System Architecture

4.3 Major System Modules

4.3.1 Authentication and Authorization Module

This module manages registration, login, password hashing, user roles, and protected API access.

It ensures that users can perform only those operations permitted by their roles and ownership permissions.

4.3.2 User Management Module

This module maintains user profiles and account status. Administrators can manage users, while students and instructors can view or update permitted profile information.

4.3.3 Course Management Module

This module enables instructors to create courses, add descriptions, define course categories, organize learning content, and publish courses.

4.3.4 Enrollment Module

This module records student enrollment in courses. It prevents duplicate enrollments and verifies that students have access to the appropriate learning resources.

4.3.5 Learning Content Module

This module organizes course content into modules and lessons. Lessons can contain text, documents, external video links, and other educational resources.

Large files should preferably be stored in object storage or a dedicated media service rather than directly in relational database records.

4.3.6 Assessment Module

This module supports quiz creation, question management, answer submission, automatic grading for supported question types, and result calculation.

4.3.7 Progress Tracking Module

This module maintains lesson completion status, quiz scores, course progress percentages, and completion history.

A course progress calculation may use:

$$\text{Progress}(\%) = \frac{\text{Completed Lessons}}{\text{Total Lessons}}\times100$$

4.3.8 Reporting Module

This module provides dashboards and reports about course participation, quiz performance, completion rates, and learner progress.

4.3.9 Container Deployment Module

This module defines Dockerfiles, image build instructions, container configuration, and Kubernetes deployment manifests.

It supports repeatable application deployment and provides a foundation for scaling and operational monitoring.

4.4 Database Design

Table 4.1 Database Design

Entity	Important Attributes	Purpose
User	user_id, name, email, password_hash, role	Stores account information
Course	course_id, title, description, instructor_id, status	Stores course details
Module	module_id, course_id, title, sequence_no	Organizes course content
Lesson	lesson_id, module_id, title, content_url, sequence_no	Stores lesson metadata
Enrollment	enrollment_id, student_id, course_id, enrolled_at	Records course enrollment
Quiz	quiz_id, course_id, title, total_marks	Stores assessment details
Question	question_id, quiz_id, question_text, marks	Stores quiz questions
Submission	submission_id, quiz_id, student_id, score, submitted_at	Records quiz attempts
LessonProgress	progress_id, student_id, lesson_id, completed	Records lesson completion

The schema should define primary keys, foreign keys, unique constraints, and indexes where appropriate. For example, the enrollment table can enforce uniqueness on the combination of `student_id` and `course_id`.

4.5 REST API Design

Table 4.2 REST API Design

HTTP Method	Endpoint	Purpose
POST	`/api/auth/register`	Register a user
POST	`/api/auth/login`	Authenticate a user
GET	`/api/courses`	List published courses
GET	`/api/courses/{id}`	Retrieve course details
POST	`/api/courses`	Create a course
PUT	`/api/courses/{id}`	Update a course
POST	`/api/courses/{id}/enroll`	Enroll a student
GET	`/api/enrollments/my-courses`	List enrolled courses
GET	`/api/courses/{id}/lessons`	Retrieve course lessons
POST	`/api/quizzes/{id}/submissions`	Submit a quiz
GET	`/api/progress/my-courses`	View learning progress

The backend must validate requests and enforce permissions even when the frontend hides unauthorized operations.

4.6 Deployment Architecture

The frontend and backend are built as Docker images and deployed to Kubernetes. Kubernetes Services provide stable networking, while Deployments manage application replicas and updates.

The database may run as a managed database service or as a carefully configured Kubernetes workload with persistent volumes and backup procedures.

A production-oriented deployment should also consider TLS termination, resource requests and limits, health probes, secrets management, logging, monitoring, and database backup and recovery.

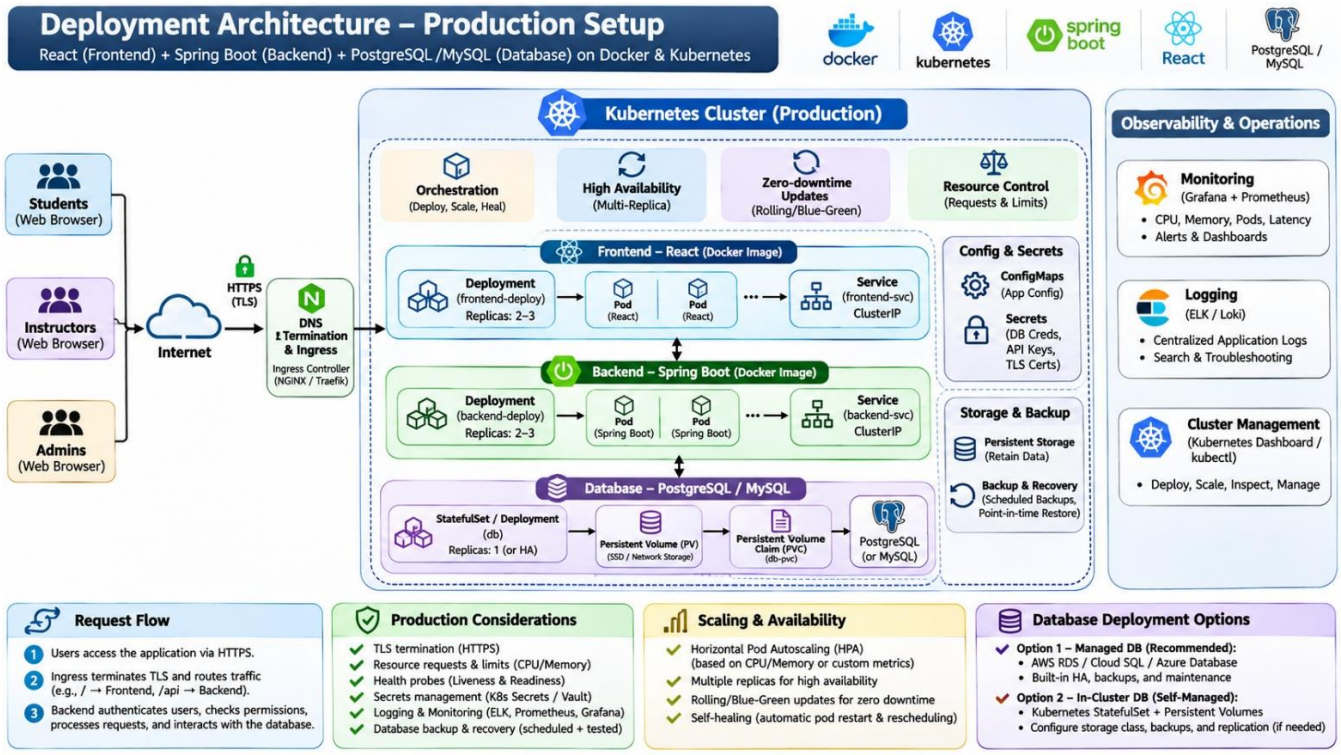


Fig 4.2 Deployment Architecture

4.7 Summary

The proposed design separates application responsibilities into well-defined modules and layers. The architecture supports future enhancements and provides the structure needed for implementation, testing, and deployment.

CHAPTER-5
IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter describes the implementation approach for the frontend, backend, database, Docker containers, and Kubernetes deployment. It also defines the testing strategy used to verify application functionality and deployment behavior.

5.2 Development Environment Setup

The development environment requires Java, Maven, Node.js, a relational database, Docker, and Kubernetes tooling.

The initial setup involves:

1. Creating a Spring Boot project with the required dependencies.
2. Creating the frontend project.
3. Configuring the database connection.
4. Defining application entities and repositories.
5. Implementing REST controllers and business services.
6. Connecting frontend screens to the backend APIs.
7. Testing application functionality locally.
8. Creating Docker images.
9. Deploying the containers to Kubernetes.

5.3 Backend Implementation

The backend follows a layered architecture.

- Controller Layer: Receives HTTP requests and returns API responses.
- Service Layer: Implements business logic and transaction management.
- Repository Layer: Provides database access using Spring Data JPA.
- Entity Layer: Represents persistent database records.
- DTO Layer: Defines request and response structures.
- Security Layer: Implements authentication and authorization.

The course enrollment workflow provides an example of the backend process:

1. The student sends an enrollment request.
2. Spring Security authenticates the user.
3. The controller validates the incoming request.
4. The service verifies the course status and student eligibility.
5. The repository checks for an existing enrollment.
6. The service saves the enrollment.
7. The backend returns an appropriate response.

Database transactions and uniqueness constraints should be used to avoid inconsistent records and duplicate enrollments.

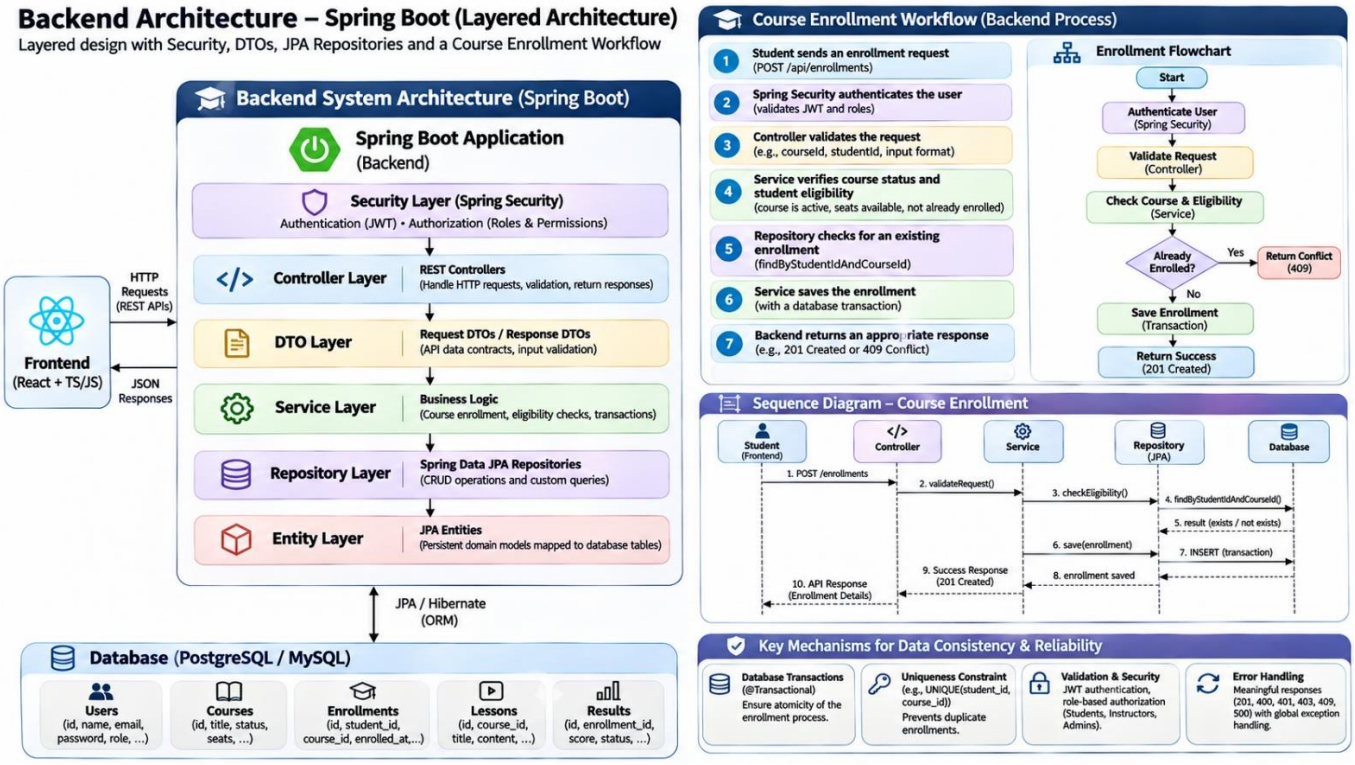


Fig 5.1 Backend Architecture

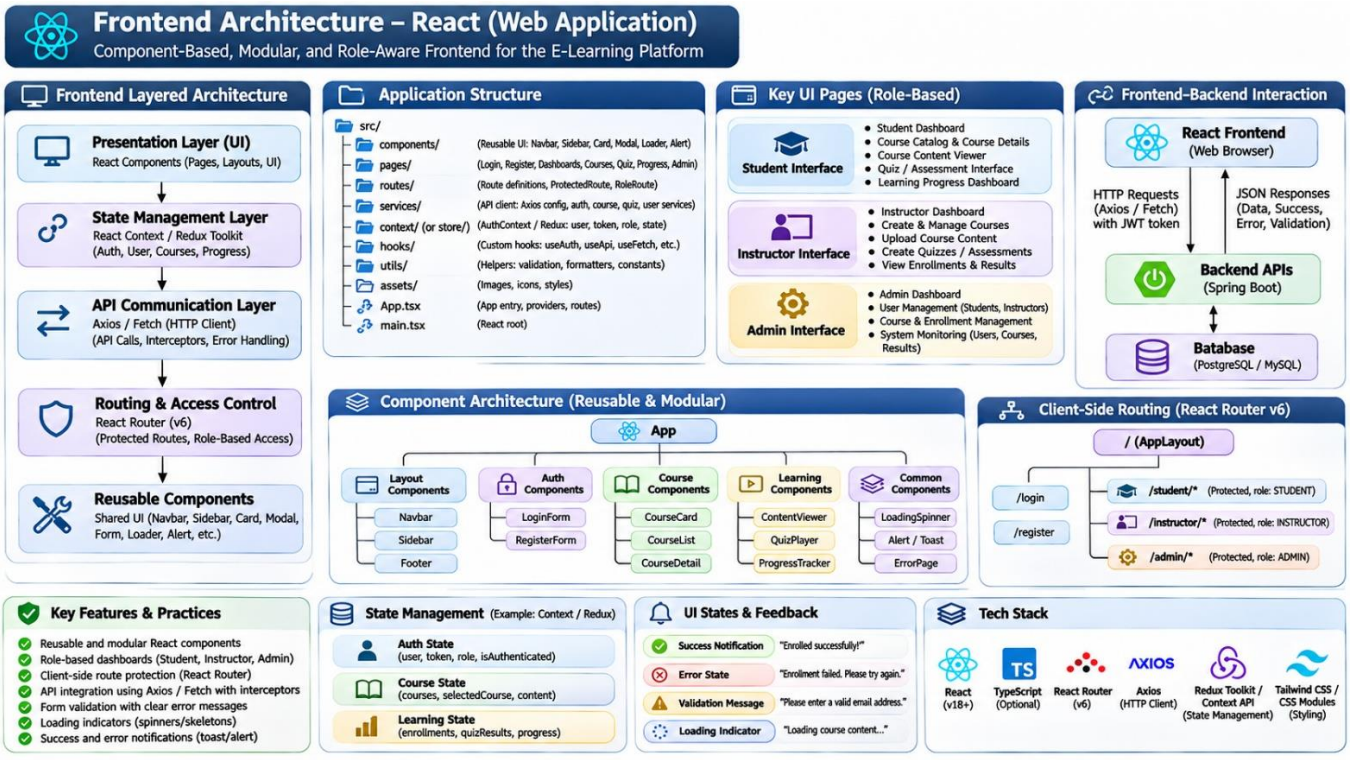
5.4 Frontend Implementation

The frontend is developed using reusable React components [4].

The principal pages include:

- Login and registration pages.
- Student dashboard.
- Instructor dashboard.
- Course catalog and course details.
- Course content viewer.
- Quiz and assessment interface.
- Learning progress dashboard.
- Administrator user-management interface.

The frontend calls backend APIs using an HTTP client such as the Fetch API or Axios. It should display validation messages, loading indicators, error states, and success notifications.



5.2 Frontend Architecture

5.5 Database Implementation

The database schema is created using entity mappings, migrations, or controlled SQL scripts.

The implementation includes:

- User and role records.
- Course and lesson records.
- Enrollment relationships.
- Assessment questions and submissions.
- Lesson completion and progress records.

Database migrations using a tool such as Flyway can help maintain consistent schema versions across environments.

Passwords must be stored as secure password hashes, not plaintext. Application credentials should be supplied through protected configuration.

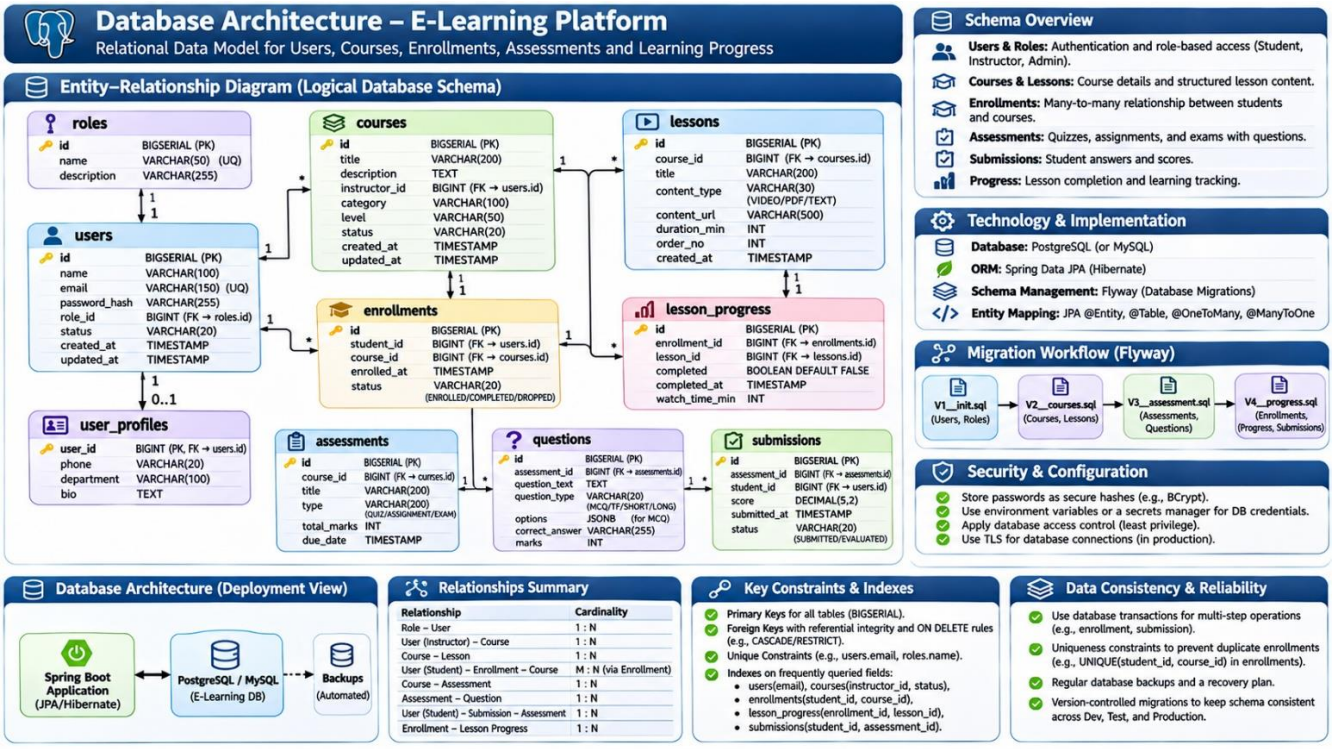


Fig 5.3 Database Architecture

5.6 Docker Implementation

Docker files are created to package the frontend and backend [7].

A typical backend container build process includes:

1. Building the Spring Boot application using Maven.
2. Packaging the resulting JAR.
3. Copying the JAR into a compatible Java runtime image.
4. Exposing the configured application port.
5. Supplying environment-specific configuration at runtime.

The frontend can be built into static assets and served through a lightweight web server container.

The database should have persistent storage and appropriate backup procedures; containerizing a database alone does not guarantee data persistence.

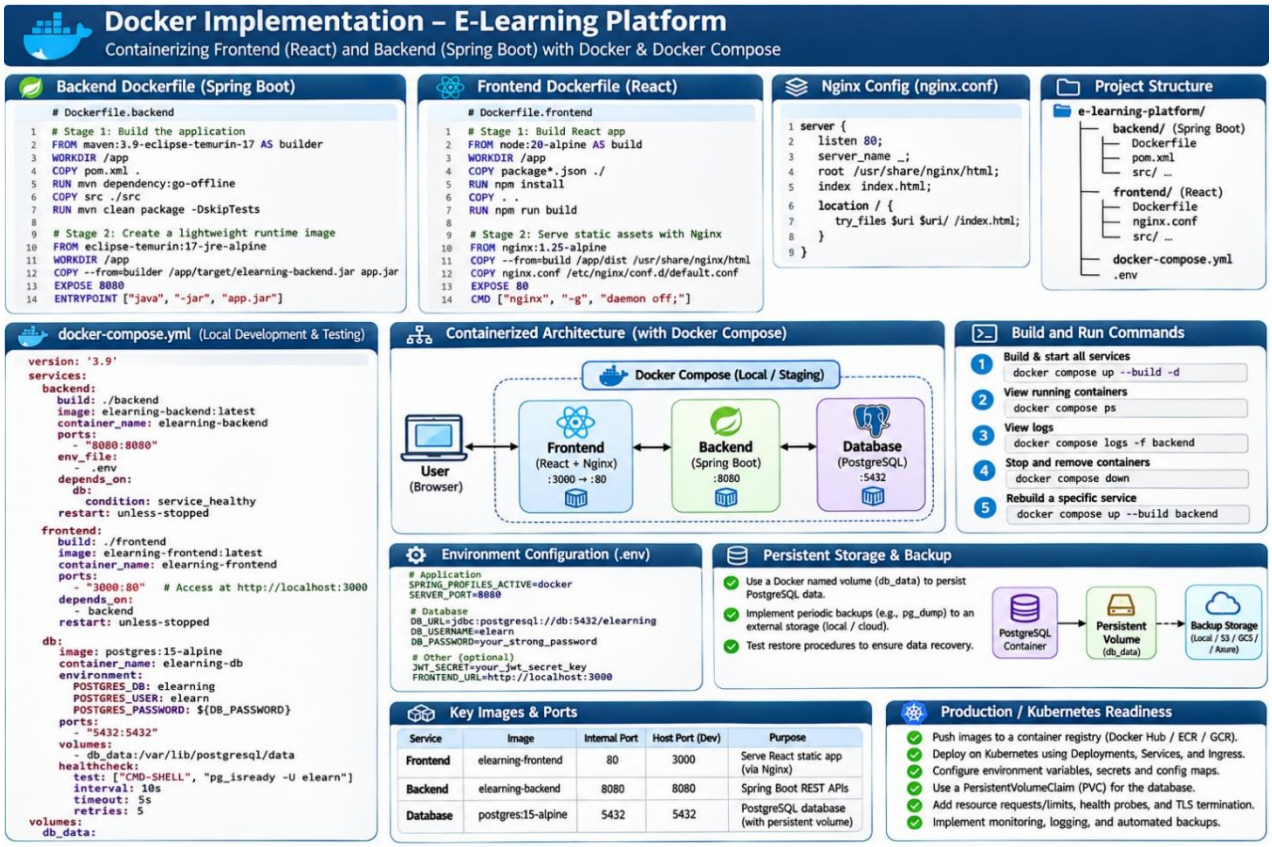


Fig 5.4 Docker Architecture

5.7 Kubernetes Implementation

Kubernetes manifests define the application deployments and their networking configuration [9].

The deployment process includes:

1. Building Docker images.
2. Publishing the images to a container registry.
3. Configuring Kubernetes access to the registry.
4. Creating configuration resources and secrets.
5. Deploying the backend and frontend.
6. Creating Kubernetes Services.
7. Configuring health probes and resource limits.
8. Verifying application connectivity.
9. Testing scaling and recovery behavior.

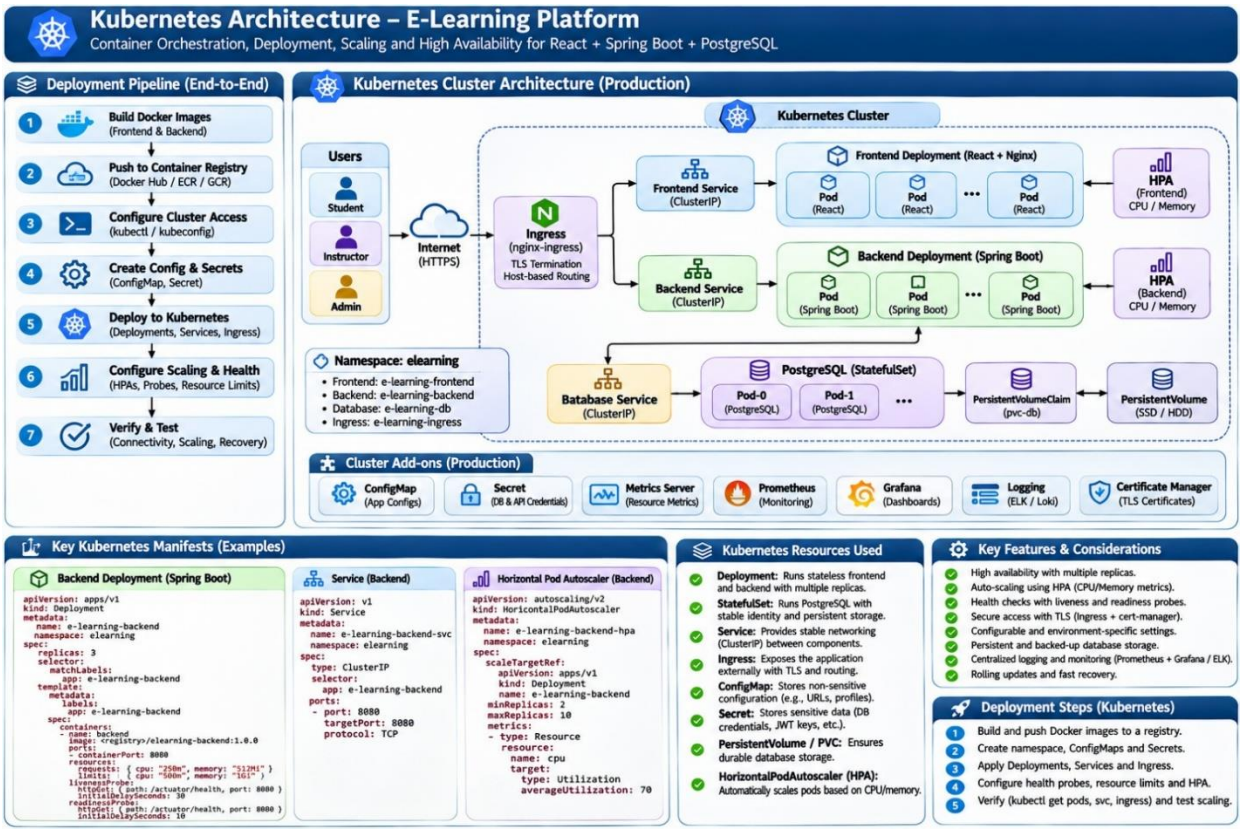


Fig 5.5 Kubernetes Architecture

5.8 Testing Strategy

The application is evaluated using multiple testing levels.

Unit Testing: Verifies individual service methods, validation rules, and utility logic.

Integration Testing: Verifies communication between Spring Boot components and the database.

API Testing: Tests REST endpoints using Postman or automated integration tests.

Frontend Testing: Checks form validation, navigation, rendering, and user interactions.

System Testing: Verifies complete user workflows across the frontend, backend, and database.

Security Testing: Checks unauthorized access, role enforcement, invalid input, and secure credential handling.

Container Testing: Verifies that Docker images build successfully and containers start correctly.

Deployment Testing: Validates Kubernetes rollout, service access, readiness, and recovery from selected container failures.

Performance Testing: Measures response time, throughput, and resource utilization under a defined workload.

5.9 Sample Test Cases

Table 5.1 Sample Test Cases

Test ID	Test Scenario	Expected Result
TC01	Register with valid details	Account is created
TC02	Register using an existing email	Request is rejected
TC03	Log in with valid credentials	User is authenticated
TC04	Log in with invalid credentials	Access is denied
TC05	Instructor creates a course	Course is saved
TC06	Student enrolls in a published course	Enrollment is recorded
TC07	Student attempts a quiz	Submission is evaluated
TC08	Student completes a lesson	Progress is updated
TC09	Unauthorized user modifies a course	Request is rejected
TC10	Backend container restarts	Workload recovers when configured correctly
TC11	Kubernetes deployment is updated	New application version is rolled out
TC12	Application is accessed through its configured service	Application responds as expected

These are test specifications. Actual results should be recorded only after the test cases are executed.

5.10 Performance and Deployment Evaluation

The following metrics can be used to evaluate the system:

- API response time.
- Requests processed per second.
- Error rate under load.
- CPU and memory utilization.
- Container restart frequency.
- Deployment success rate.
- Database query performance.
- Learning workflow completion rate.

Load tests should use a documented workload and environment. Kubernetes scaling behavior should be evaluated under controlled conditions rather than assumed from configuration alone.

5.11 Summary

This chapter describes the implementation workflow and testing methodology. The combination of functional testing, security checks, container verification, and deployment evaluation helps determine whether the platform satisfies its requirements.

CHAPTER-6
RESULTS AND FUTURE ENHANCEMENTS

6.1 Introduction

This chapter describes the intended outcomes of the project, explains how the system can be evaluated, summarizes its contributions, and identifies potential future improvements.

6.2 Expected Results

The completed platform is expected to provide a centralized web-based learning environment for students, instructors, and administrators.

The expected outcomes include:

1. A responsive user interface for accessing courses and learning materials.
2. Secure registration and role-based access control.
3. Course creation and enrollment functionality.
4. Online quizzes and assessment result management.
5. Automated recording of lesson completion and learning progress.
6. REST API communication between the frontend and backend.
7. Repeatable application packaging using Docker.
8. Kubernetes-based deployment and management of application containers.
9. Support for scaling application replicas when resources and configuration permit.
10. Improved maintainability through separation of application responsibilities.

These outcomes represent project objectives and should be confirmed through actual implementation and testing.

6.3 Result Evaluation

The project should be evaluated against its functional and operational requirements.

Table 6.1 Result Evaluation

Evaluation Area	Measurement or Evidence
Authentication	Successful and unsuccessful login tests
Course management	Verified create, update, publish, and retrieval operations
Enrollment	Correct enrollment records and duplicate prevention
Assessment	Correct submission and grading behavior
Progress tracking	Correct completion counts and progress percentages
API functionality	API test results and response codes
Security	Results of authorization and input-validation tests
Docker deployment	Successful image builds and container startup
Kubernetes deployment	Successful rollout and service connectivity
Resilience	Observed recovery during controlled container failure tests
Performance	Measured latency, throughput, and resource consumption

Measured performance values should be included in the final report only after running tests and collecting results.

6.4 Project Contributions

The project demonstrates how a full-stack educational application can be designed using modern application development and deployment technologies.

Its major contributions include:

- Integrating frontend and backend development into one learning solution.
- Applying layered architecture and RESTful API design.
- Implementing structured course and assessment management.
- Maintaining student learning progress in a relational database.
- Using Docker to create consistent deployment artifacts.
- Using Kubernetes to manage containerized application workloads.
- Establishing a foundation for future cloud-native learning services.

6.5 Limitations

The initial version may have several limitations:

- Learning content may be limited to text, documents, and external video links.
- Advanced live-class and real-time collaboration features may not be included.
- Automatic grading may support only selected question types.
- Personalized recommendations may not be available.
- Application scaling does not automatically eliminate database bottlenecks.
- Kubernetes availability depends on cluster configuration, infrastructure capacity, and persistent storage.
- Advanced analytics may require additional data processing and reporting components.

These limitations can be addressed through phased development and further evaluation.

6.6 Future Enhancements

- Machine learning techniques can analyze learner activity, assessment performance, and course completion history to recommend relevant courses and learning materials.
- WebRTC or a suitable conferencing integration can support live sessions, screen sharing, and interactive discussions.
- As the platform grows, selected capabilities such as authentication, course management, assessment, notifications, and reporting can be separated into independently deployable services where justified by operational needs.
- A CI/CD pipeline can automate testing, Docker image creation, security scanning, and Kubernetes deployment using tools such as Jenkins or GitHub Actions.
- Prometheus and Grafana can provide metrics and dashboards. Centralized logging and distributed tracing can help identify application errors and performance bottlenecks.
- The application can be deployed to a cloud platform using managed Kubernetes, managed relational databases, object storage, and cloud monitoring services.
- The platform can generate course completion certificates after verifying eligibility and course completion requirements.
- Badges, points, learning streaks, and leaderboards can encourage learner participation when implemented with appropriate privacy and fairness considerations.

CONCLUSION

The Full Stack Online Learning Platform Using Spring Boot, Docker and Kubernetes propose an integrated solution for digital education and course management. The system combines a modern frontend, Spring Boot REST APIs, relational data persistence, and containerized deployment to provide a structured learning environment.

The platform addresses key requirements such as user management, course creation, enrollment, learning content delivery, assessment, and progress tracking. Docker supports consistent application packaging, while Kubernetes provides mechanisms for workload management, service discovery, rolling updates, and scaling.

The project also provides practical exposure to full-stack application development, REST API design, database integration, containerization, orchestration, and software testing. With further enhancements in monitoring, automation, cloud deployment, and AI-driven personalization, the platform can evolve into a more comprehensive and scalable learning management solution.

Program Outcomes Achieved

PO1, PO2, PO3, PO4, PO5, PO8, PO9, PO10.

REFERENCES

[1] Broadcom Inc., “Spring Boot Reference Documentation.” [Online]. Available: https://docs.spring.io/spring-boot/index.html (accessed Oct. 24, 2026).

[2] Broadcom Inc., “Spring Data JPA Reference Documentation.” [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/ (accessed Oct. 24, 2026).

[3] Broadcom Inc., “Spring Security Reference.” [Online]. Available: https://docs.spring.io/spring-security/reference/ (accessed Oct. 24, 2026).

[4] Meta Platforms, Inc., “React Documentation.” [Online]. Available: https://react.dev/ (accessed Oct. 24, 2026).

[5] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing React Apps*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2020.

[6] The PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: https://www.postgresql.org/docs/ (accessed Oct. 24, 2026).

[7] Docker Inc., “Docker Documentation.” [Online]. Available: https://docs.docker.com/ (accessed Oct. 24, 2026).

[8] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, Mar. 2014.

[9] The Kubernetes Authors, “Kubernetes Documentation.” [Online]. Available: https://kubernetes.io/docs/ (accessed Oct. 24, 2026).

[10] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.